Complex Lane Packing for Encrypted GQA Key/Value Caches

Cachemir GQA – Ciphertext Pipeline

July 14, 2026

Abstract

This note documents a lane-packing optimization applied to the Orion-based ciphertext implementation of Grouped Query Attention (GQA) decoding in `Cachemir/gqa/ciphertext`. CKKS ciphertext slots natively encode complex numbers, but the existing pipeline only ever populated the real component (imaginary part fixed at zero), discarding half of the available packing density. We introduce a scheme that packs two tokens per ciphertext slot – one in the real part, one in the imaginary part – for the Key/Value cache, and show how the resulting cross-term contamination in ciphertext–ciphertext multiplication is avoided (for $QK^\top$) or removed algebraically using complex conjugation (for scores$\times V$). We give the exact algorithms, a correctness proof of the conjugate trick, and measured before/after cost tables.

Contents

1 Background **2**
- 1.1 CKKS slots and the packing gap . . . 2
- 1.2 GQA slot layout (recap) . . . 2

2 Complex Lane Packing Scheme **2**
- 2.1 Idea . . . 2
- 2.2 Writing packed tokens: extending `vmm_kv` . . . 3
- 2.3 Cache capacity doubling . . . 3

3 Consuming Packed Ciphertexts in Attention **3**
- 3.1 $QK^\top$: packing is "free" because Q stays real . . . 3
- 3.2 scores$\times V$: the conjugate trick . . . 4
- 3.3 The missing primitive: homomorphic conjugation . . . 4

4 Algorithms **5**

5 Cost Model **6**

6 Measured Results **6**
- 6.1 Cache ciphertext count (memory) . . . 6
- 6.2 Ciphertext–ciphertext multiplications ($QK^\top$ + scores$\times V$) . . . 6
- 6.3 Rotations and other operation counts . . . 7
- 6.4 Correctness . . . 7

7 Benchmark: Complex vs. Real-Only, Subprocess-Isolated **7**
7.1 Hardware 7
7.2 Results 8
7.3 Discussion 9

1 Background

1.1 CKKS slots and the packing gap

A CKKS ciphertext with n_{he} slots natively encodes a vector in $\mathbb{C}^{n_{he}}$: every slot holds one *complex* number, and ciphertext–ciphertext and ciphertext–plaintext multiplication are defined slot-wise on $\mathbb{C}$. The existing GQA pipeline only ever encodes real-valued tensors (`torch.float64`), i.e. every slot is used as $z = a + 0i$. The imaginary half of every slot's capacity is therefore completely unused – a factor-of-two packing inefficiency that propagates directly into ciphertext count (memory) and into the number of ciphertext–ciphertext (ct-ct) multiplications required to consume the Key/Value (K/V) cache, which is the dominant cost of encrypted attention.

1.2 GQA slot layout (recap)

Let n_{he} be the number of CKKS slots, d_{kv} the compact K/V feature width, H the number of query heads, n_{kv} the number of K/V heads, ratio $= H/n_{kv}$, and

$$t_p = \frac{n_{he}}{d_{kv}}$$

the number of tokens that fit, *before* this optimization, in a single real-valued cache ciphertext (one token occupies every t_p-th slot, i.e. physical lane $\ell \in \{0, \dots, t_p - 1\}$ selects slots $\{\ell, \ell + t_p, \ell + 2t_p, \dots\}$). A cache of N tokens therefore requires

$$C_{\text{before}}(N) = \left\lceil \frac{N}{t_p} \right\rceil$$

ciphertexts, and the attention read-side loops ('$QK^\top$' and 'scores$\times V$') iterate once per cache ciphertext, each iteration costing one ct-ct multiplication.

2 Complex Lane Packing Scheme

2.1 Idea

Because each CKKS slot already holds a complex number, we can pack a *second*, independent token into the previously-unused imaginary component of the same physical lane. Concretely, token $2k$ is written into the real part of lane ℓ and token $2k + 1$ into the imaginary part of the *same* lane ℓ:

$$\text{slot}(\ell) \;=\; \underbrace{x_{2k}}_{\Re} \;+\; \underbrace{x_{2k+1}}_{\Im}\, i.$$

This doubles the effective per-ciphertext capacity to $2t_p$ tokens, so a cache of N tokens now needs only

$$C_{\text{after}}(N) = \left\lceil \frac{N}{2t_p} \right\rceil \;\approx\; \tfrac{1}{2}\, C_{\text{before}}(N)$$

ciphertexts.

2.2 Writing packed tokens: extending `vmm_kv`

The projection routine `vmm_kv` writes one new token into a cache ciphertext at structural position pos using a plaintext mask $m_{\text{pos}} \in \{0,1\}^{n_{he}}$ with $m_{\text{pos}}[\ell] = \mathbb{1}[\ell \equiv \text{pos} \pmod{t_p}]$, via masked $= (X_c W_c) \odot m_{\text{pos}}$ (a ct-pt multiply), after folding replicated contributions into the correct lane with $\log_2 t_p$ rotations. We extend pos to range over $\{0, \ldots, 2t_p - 1\}$:

- pos $< t_p$: unchanged – real mask, contributes to $\Re$(slot).
- pos $\geq t_p$: physical lane $\ell = \text{pos} - t_p$, but the mask is multiplied by the imaginary unit,

$$m'_{\text{pos}} = i \cdot m_\ell \in \{0, i\}^{n_{he}},$$

  so the ct-pt product lands in $\Im$(slot) instead of $\Re$(slot).

No other change is required: rotation and addition act identically (and independently) on the real and imaginary components of a slot, since both are $\mathbb{C}$-linear operations.

2.3 Cache capacity doubling

The packed-cache classes (`HEKCache`/`HEVCache`) already accumulate tokens into the *current* ciphertext until a capacity threshold is reached, then start a new one. Doubling the threshold,

$$t \leftarrow 2t_p,$$

is sufficient: the existing append logic transparently drives the extended pos range above, alternating between the real half (pos $\in [0, t_p)$) and the imaginary half (pos $\in [t_p, 2t_p)$) of each ciphertext.

3 Consuming Packed Ciphertexts in Attention

3.1 $QK^\top$: packing is "free" because Q stays real

For two complex numbers $a = a_r + a_i i$ and $b = b_r + b_i i$, multiplication mixes components:

$$ab = (a_r b_r - a_i b_i) + (a_r b_i + a_i b_r)\, i.$$

In general this means multiplying two lane-packed ciphertexts contaminates each output lane with cross terms from *both* packed values – exactly the failure mode we must avoid.

However, in `qkt_gqa_he` the query ciphertext Q is *never* lane-packed: it is produced once per decoding step from a single new token via pos $= 0$ only, so $Q = q + 0i$ is purely real ($a_i = 0$). Substituting $a_i = 0$ above,

$$Q \cdot K_{\text{pack}} \;=\; q\,(k_r + k_i i) \;=\; \underbrace{qk_r}_{\Re} \;+\; \underbrace{qk_i}_{\Im}\, i,$$

which has *no* cross term: the real part is exactly the score against the even-indexed cached token and the imaginary part is exactly the score against the odd-indexed cached token. One ct-ct multiplication against a packed K ciphertext therefore yields two independent, uncontaminated scores – a $2\times$ reduction in ct-ct multiplications for this stage at no extra algorithmic cost.

3.2 scores$\times V$: the conjugate trick

The second stage, `softmax_v_gqa_he`, multiplies the (now complex, paired) attention scores $s = s_r + s_i i$ against the packed value cache $V = v_r + v_i i$. Here *both* operands are complex, so a direct product does mix lanes:

$$sV = (s_r v_r - s_i v_i) + (s_r v_i + s_i v_r)i.$$

What we actually want to accumulate into the running output is the sum of both paired tokens' contributions,

$$s_r v_r + s_i v_i,$$

with no cross term. This is obtained by multiplying by the *complex conjugate* of V instead of V itself. Since $\overline{V} = v_r - v_i i$,

$$s \cdot \overline{V} = (s_r + s_i i)(v_r - v_i i) = \underbrace{(s_r v_r + s_i v_i)}_{\Re} + \underbrace{(s_i v_r - s_r v_i)}_{\Im} i.$$

The real part is *exactly* the desired paired-token accumulation; the imaginary part is discardable cross-term noise. Because the final client-side decode already keeps only $\Re(\cdot)$ (CKKS decoding always discards residual imaginary noise), no additional homomorphic real-part extraction is needed – a single ct-ct multiplication $s \cdot \overline{V}$ replaces what would otherwise require two separate multiplications (one per unpacked token).

3.3 The missing primitive: homomorphic conjugation

Complex conjugation of a CKKS ciphertext is implemented as a Galois automorphism (the same family of operation used for slot rotations, but with a different Galois element – the order-2 element of the multiplicative group associated with the ring, sometimes called the "orthogonal subgroup" element). The underlying Lattigo library exposes this natively as `Evaluator.Conjugate`, but the Orion wrapper used by this pipeline did not expose it. We added it end-to-end:

- **Go layer** (`evaluator.go`): new exported `Conjugate`/`ConjugateNew` functions, plus `AddConjugationKey()` which generates (once, lazily) the Galois key for the conjugation automorphism, mirroring the existing power-of-two rotation key cache.
- **C bindings** (`bindings.py`): registered the two new exported symbols.
- **Python evaluator** (`evaluator.py`): a `conjugate(ctxt, in_place)` wrapper mirroring `rotate(...)`.
- **Tensor API** (`tensors.py`): a `CipherTensor.conjugate()` method mirroring `CipherTensor.roll()`.

This primitive was validated in isolation: encrypting a random complex vector, conjugating homomorphically, decrypting, and comparing against `numpy.conj` gave a maximum absolute error of $\approx 3.4 \times 10^{-8}$, consistent with ordinary CKKS rescaling noise.

4 Algorithms

Algorithm 1 `vmm_kv` with paired real/imaginary positions

Require: encrypted sparse chunks $\{X_c\}$, plaintext weight chunks $\{W_c\}$, dims (incl. t_p), level ℓ, position pos $\in [0, 2t_p)$

is_imag $\leftarrow$ (pos $\geq t_p$)
phys $\leftarrow$ pos $- t_p \cdot \mathbb{1}$[is_imag]
$m \leftarrow \mathbf{0} \in \mathbb{C}^{n_{he}}$
m[phys :: t_p] $\leftarrow i$ **if** is_imag **else** 1
out $\leftarrow \varnothing$
for each (X_c, W_c) **do**
    acc $\leftarrow X_c \cdot W_c$ ▷ ct-pt multiply
    for step $= 1, 2, 4, \cdots < t_p$ **do** ▷ $\log_2 t_p$ rotations
        acc $\leftarrow$ acc + roll(acc, $\pm$step) ▷ sign chosen from bits of phys
    end for
    masked $\leftarrow$ acc $\odot\, m$ ▷ ct-pt multiply (complex m if imaginary half)
    out $\leftarrow$ out + masked
end for
return out

Algorithm 2 Packed $QK^\top$ (`qkt_gqa_he`)

Require: real query ciphertext Q (per group c), packed key ciphertexts $\{K_b\}_{b=1}^{C_{\text{after}}(N)}$

$Q_{\text{rep}} \leftarrow$ BlockReplicate(Q, t_p)
for each packed key ciphertext K_b **do**
    folded $\leftarrow Q_{\text{rep}} \cdot K_b$ ▷ 1 ct-ct mult scores *two* tokens; no cross terms since Q_{rep} is real
    fold folded over $\log_2 d_h$ rotations (head-dim reduction)
    append folded to attention row for group c
end for

Algorithm 3 Packed scores$\times V$ (`softmax_v_gqa_he`)

Require: packed attention scores $\{s_b\}$, packed value ciphertexts $\{V_b\}$

out $\leftarrow 0$
for each b **do**
    $\overline{V_b} \leftarrow$ Conjugate(V_b) ▷ new automorphism
    prod $\leftarrow s_b \cdot \overline{V_b}$ ▷ 1 ct-ct mult; $\Re(\text{prod}) = s_r v_r + s_i v_i$
    fold prod over $\log_2 t_p$ rotations
    out $\leftarrow$ out + prod $\odot$ out_mask ▷ ct-pt multiply, real mask
end for
return out ▷ client discards $\Im(\cdot)$ on decode

5 Cost Model

Let ratio $= H/n_{kv}$ and $C(N)$ be the number of K/V cache ciphertexts for N cached tokens. Per decoding step, the read-side cost of $QK^\top$ and scores$\times V$ combined is

$$\#\text{ct-ct} \;=\; 2 \cdot \text{ratio} \cdot C(N), \qquad \#\text{conjugations} \;=\; \begin{cases} 0 & \text{before (n/a)} \\ \text{ratio} \cdot C(N) & \text{after} \end{cases}$$

with $C_{\text{before}}(N) = \lceil N/t_p \rceil$ and $C_{\text{after}}(N) = \lceil N/(2t_p) \rceil$. The write-side cost (new token projection via `vmm_kv`, dominated by $R \cdot \text{ratio} \cdot \log_2 t_p$ rotations per token, where $R = \lceil d_{kv}/t_p \rceil$) is *unchanged* by this optimization, since each decoding step still injects exactly one new token regardless of packing.

6 Measured Results

Configuration: $n_{he} = 512$, $d = 128$, $H = 8$, $n_{kv} = 4$ ($d_h = 16$, ratio $= 2$, $d_{kv} = 64$, $t_p = 8$).

6.1 Cache ciphertext count (memory)

Tokens N	Before $\lceil N/t_p \rceil$	After $\lceil N/2t_p \rceil$	Reduction
6	1	1	0%
21	3	2	33%
51	7	4	43%
101	13	7	46%

Table 1: K/V cache ciphertext count before/after complex packing. Very short sequences see no benefit (a single ciphertext already covers them either way); the reduction approaches the asymptotic 2× as N grows.

6.2 Ciphertext–ciphertext multiplications ($QK^\top$ + scores$\times V$)

Tokens N	Before	After (measured)	Reduction	Conjugations (after)
6	4	4	0%	2
21	12	8	33%	4
51	28	16	43%	8
101	52	28	46%	14

Table 2: ct-ct multiplication counts. "Before" values are computed from the closed-form cost model above (consistent with the pre-packing code path); "after" values are measured directly from the instrumented pipeline. Conjugations are a new op class introduced by this optimization, each cheaper than a ct-ct multiplication.

6.3 Rotations and other operation counts

Tokens N	Rotations	ct-pt mult	ct-ct mult	Conjugations
6	1448	450	4	2
21	4792	1412	8	4
51	11480	3336	16	8
101	22622	6542	28	14

Table 3: Full measured counter dump for the packed ("after") pipeline. Rotation counts are essentially unchanged from the pre-packing baseline at matching N: they are dominated by the per-token write path in `vmm_kv` (folding each new token into position, unaffected by packing), not by the read-side loops that benefit from packing. Only the smaller $QK^\top$/scores$\times V$ fold-rotation contributions shrink proportionally to the ct-ct savings above.

6.4 Correctness

Tokens N	qkt_err	softmaxv_err	Status
6	7.16×10^{-5}	2.93×10^{-3}	CHECK
21	7.16×10^{-5}	4.32×10^{-3}	CHECK
51	1.05×10^{-4}	7.22×10^{-3}	CHECK
101	1.05×10^{-4}	1.09×10^{-2}	CHECK

Table 4: Errors against the (unchanged, real-valued) plaintext reference. The N=6 case matches the pre-existing, documented CKKS approximation baseline for this configuration exactly (no regression from packing); the slow growth with N reflects accumulating CKKS noise across more ct-ct terms, independent of whether packing is used.

7 Benchmark: Complex vs. Real-Only, Subprocess-Isolated

The cost-model tables in Section 6 were derived from operation counters within a single long-lived process. To also obtain clean, per-mode latency and peak-memory numbers, a dedicated benchmark harness (`Cachemir/gqa/ciphertext/benchmark_complex_packing.py`) runs each configuration *twice*, once with `pack_complex=True` and once with `pack_complex=False`, in two separate, freshly-spawned subprocesses per configuration. This isolation is necessary because peak resident set size (RSS) is a monotonically non-decreasing, process-wide high-water mark (via `resource.getrusage(RUSAGE_SELF).ru_maxrss`): measuring it for two modes inside the same process would contaminate the second measurement with the first mode's memory usage.

7.1 Hardware

Benchmarks were run on a single (non-virtualized) machine with the following relevant specifications:

- CPU: 2× Intel(R) Xeon(R) Platinum 8260 @ 2.40 GHz (24 physical cores per socket, 48 physical cores / 96 logical threads total via hyperthreading), NUMA with 2 nodes.
- Cache: 1.5 MiB L1d + 1.5 MiB L1i, 48 MiB L2, 71.5 MiB L3 (shared, 2 instances).

- Memory: 250 GiB RAM.
- OS/kernel: Linux 6.8 (Ubuntu), x86_64.

Despite the machine having 96 logical cores available, the pipeline as benchmarked is *single-threaded at the application level*: the Orion/Lattigo Go backend invoked here performs each ciphertext operation (rotation, ct-pt/ct-ct multiplication, conjugation) as one sequential call from the Python driver, and no goroutine/thread-pool parallelism across independent ciphertext operations is used in this code path (the Orion backend wrapper contains no explicit concurrency primitives such as `go func(...)` or worker pools for these operations). The reported wall-clock numbers therefore reflect single-core throughput of the CKKS backend, not a parallel/multi-core execution; the large core count and RAM headroom of the host mean the measurements are not resource-constrained, but they also do not benefit from multi-core speedup, and results should scale similarly on any single fast core of comparable clock speed.

7.2 Results

Each configuration below ran an n_{prefill}-token prefill loop followed by one encrypted decoding step, using $n_{he} = 512$, $d = 128$, $H = 8$, $n_{kv} = 4$ ($t_p = 8$, ratio = 2) for the two larger cases, and $d = 8$, $H = 4$, $n_{kv} = 2$ ($t_p = 128$) for the smallest case.

Metric (d=8, n_{prefill}=5)	Complex	Real-only	Ratio
Latency: prefill (s)	0.803	0.813	0.99×
Latency: decode step (s)	0.420	0.404	1.04×
Latency: total (s)	1.268	1.262	1.00×
Peak RSS (MB)	857.0	851.0	1.01×
kcache / vcache ciphertexts	1 / 1	1 / 1	1.00×
att_cts / O ciphertexts	2 / 2	2 / 2	1.00×
ct-ct mult (total)	4	4	1.00×
Conjugations (total)	2	0	n/a

Table 5: Smallest case: cache already fits in a single ciphertext under both layouts (t_p=128 $\gg$ N=6 tokens), so packing has no effect – included as a control.

Metric (d=128, n_{prefill}=9)	Complex	Real-only	Ratio
Latency: prefill (s)	9.334	9.293	1.00×
Latency: decode step (s)	1.147	1.181	0.97×
Latency: total (s)	11.030	11.026	1.00×
Peak RSS (MB)	1598.0	1600.2	1.00×
kcache / vcache ciphertexts	1 / 1	2 / 2	0.50×
att_cts ciphertexts	2	4	0.50×
O ciphertexts	2	2	1.00×
ct-ct mult (total)	4	8	0.50×
ct-ct mult ($QK^\top$ phase)	2	4	0.50×
ct-ct mult (scores×V phase)	2	4	0.50×
Conjugations (total)	2	0	n/a

Table 6: $N = 10$ tokens is just past the real-only layout's single-ciphertext capacity (t_p=8), so the cache splits into 2 ciphertexts real-only vs. 1 under packing – the ct-ct multiplication count halves accordingly in both the $QK^\top$ and scores×V phases.

Metric (d=128, n_{prefill}=20)	Complex	Real-only	Ratio
Latency: prefill (s)	19.742	19.843	0.99×
Latency: decode step (s)	1.205	1.256	0.96×
Latency: total (s)	21.498	21.649	0.99×
Peak RSS (MB)	2312.0	2311.3	1.00×
kcache / vcache ciphertexts	2 / 2	3 / 3	0.67×
att_cts ciphertexts	4	6	0.67×
O ciphertexts	2	2	1.00×
ct-ct mult (total)	8	12	0.67×
ct-ct mult ($QK^\top$ phase)	4	6	0.67×
ct-ct mult (scores×V phase)	4	6	0.67×
Conjugations (total)	4	0	n/a

Table 7: $N = 21$ tokens: real-only needs $\lceil 21/8 \rceil = 3$ cache ciphertexts vs. $\lceil 21/16 \rceil = 2$ under packing, matching the 0.67× ct-ct multiplication and ciphertext-count ratios observed elsewhere in this report for the same N.

7.3 Discussion

The ct-ct multiplication and ciphertext-count reductions match the closed-form cost model exactly (Section 6). However, at these configurations *total wall-clock latency and peak RAM are essentially unchanged* between the two modes ($\leq$4% difference, within run-to-run noise): the per-token *write* path (`vmm_kv`'s ct-pt multiply and $\log_2 t_p$-step rotation fold for each new token during prefill) dominates total time end-to-end, and that path is, by design, unaffected by this optimization (Section 5). The benefit is concentrated in the smaller *read*-side $QK^\top$/scores×V phases, whose ct-ct multiplication counts drop exactly as predicted; as sequence length grows well beyond the prefill regime tested here (i.e. many decoding steps re-reading an already-long cache, rather than one write per read as in this benchmark), the read-side savings are expected to dominate overall latency and peak memory far more visibly than in these prefill-dominated measurements.

- Two tokens are packed per CKKS ciphertext slot using the real and imaginary components, exploiting slot capacity that was previously always zero-padded.
- The query path is deliberately left real/unpacked, which makes the $QK^\top$ stage benefit "for free" – no cross-term contamination, no extra operations.
- The scores×V stage requires one new primitive, homomorphic complex conjugation, which was added to the Orion/ Lattigo backend; the conjugate trick algebraically isolates the desired sum of paired-token contributions in the real part of a single ct-ct product.
- Net effect: K/V cache memory and ct-ct multiplication count (the dominant FHE cost) trend toward a 2× improvement as sequence length grows, at the cost of one cheap conjugation operation per packed value ciphertext consumed. Rotation counts are essentially unaffected, since they are dominated by the per-token write path, which lies outside this optimization's scope. No regression in numerical correctness was observed.